

PROJECT TECH GUIDE

PURPOSE

This project is a mobile developer Q&A app called Nexus. It lets users sign up, ask coding questions, post answers, comment, vote, manage profiles, receive notifications, and use an AI code explainer.

This guide is meant to help you answer questions like:

- What tech stack is being used?
- How does the app work end to end?
- Which file controls a specific screen or button?
- Where should I look if I need to change a feature?

HIGH-LEVEL ARCHITECTURE

The codebase has 2 main parts:

1. mobile

- Expo + React Native app
- Handles UI, navigation, forms, local session storage, notifications state, theme state
- Calls the backend through fetch wrappers in mobile/lib/forum-api.ts

2. backend

- Node.js + Express API
- Uses MongoDB with Mongoose for app data
- Uses Appwrite for auth-related flows like OAuth, magic links, password reset
- Uses AI providers for code explanation
- Stores business logic in controllers, models, services, and middleware

TECH STACK

MOBILE STACK

- React Native
- Expo
- Expo Router for file-based routing
- TypeScript
- AsyncStorage for persisted auth/theme state
- Expo Image for remote/local images
- Expo Image Picker for profile/banner uploads
- React Navigation under Expo Router

Main config and entry files:

- mobile/package.json
- mobile/app/_layout.tsx
- mobile/app/(tabs)/_layout.tsx

- mobile/lib/auth.tsx
- mobile/lib/forum-api.ts
- mobile/lib/appearance.tsx
- mobile/lib/notifications.tsx

BACKEND STACK

- Node.js
- Express
- Mongoose
- MongoDB
- node-appwrite
- OpenAI / Groq / Gemini fallback support
- dotenv
- morgan
- cors

Main backend entry files:

- backend/server.js
- backend/src/routes/index.js
- backend/src/config/db.js
- backend/src/config/env.js
- backend/src/config/appwrite.js

DATA AND EXTERNAL SERVICES

MONGODB

Used for app data such as:

- users
- questions
- answers
- comments
- votes
- notifications
- sessions

Main model files:

- backend/src/models/User.js
- backend/src/models/Question.js
- backend/src/models/Answer.js
- backend/src/models/Comment.js
- backend/src/models/Vote.js
- backend/src/models/Notification.js
- backend/src/models/AuthSession.js

APPWRITE

Used for:

- OAuth login
- magic link login
- password reset / recovery
- remote user account coordination

Main files:

- backend/src/config/appwrite.js
- backend/src/controllers/auth.controller.js
- mobile/app/auth.tsx

CLOUDINARY

Used for profile avatar and banner image uploads from the mobile app.

Main file:

- mobile/app/edit-profile.tsx

Backend cleanup of replaced Cloudinary assets happens in:

- backend/src/controllers/auth.controller.js

AI PROVIDERS

Used for the code explainer feature.

Main files:

- mobile/app/(tabs)/explore.tsx
- backend/src/controllers/ai.controller.js
- backend/src/services/ai.service.js

Provider order:

1. Groq if configured
2. OpenAI if configured
3. Gemini if configured
4. Fallback structured response if no provider is available

FRONTEND ROUTING MAP

Expo Router uses the file structure inside mobile/app.

MAIN APP SHELL

- mobile/app/_layout.tsx

This wraps the app in:

- AppearanceProvider
- AuthProvider
- NotificationsProvider

It also defines the main stack routes like:

- auth
- reset-password
- (tabs)
- profile
- questions/[questionId]
- edit-profile
- ai-history
- admin

BOTTOM TAB AREA

- mobile/app/(tabs)/_layout.tsx

Important tab screens:

- mobile/app/(tabs)/index.tsx -> Home feed
- mobile/app/(tabs)/explore.tsx -> Ask question + AI explain
- mobile/app/(tabs)/inbox.tsx -> Notifications
- mobile/app/(tabs)/more.tsx -> Settings / quick links / sign out

OTHER SCREENS

- mobile/app/auth.tsx -> Login, register, OAuth, magic link, forgot password
- mobile/app/profile.tsx -> User profile, recent questions, recent answers
- mobile/app/edit-profile.tsx -> Edit profile, upload avatar/banner, change password
- mobile/app/admin.tsx -> User/admin management
- mobile/app/questions/[questionId].tsx -> Question detail thread, answers, comments, voting
- mobile/app/ai-history.tsx -> AI explanation history

BACKEND STRUCTURE

ENTRY POINT

- backend/server.js

What it does:

- loads Express
- enables CORS
- parses JSON
- adds logging
- mounts /api routes
- starts the server after MongoDB connection

ROUTE REGISTRATION

- backend/src/routes/index.js

Mounted modules:

- /api/auth
- /api/questions

- /api/comments
- /api/notifications
- /api/votes
- /api/ai

CONTROLLERS

- backend/src/controllers/auth.controller.js
- backend/src/controllers/questions.controller.js
- backend/src/controllers/answers.controller.js
- backend/src/controllers/comments.controller.js
- backend/src/controllers/votes.controller.js
- backend/src/controllers/notifications.controller.js
- backend/src/controllers/ai.controller.js

SERVICES

- backend/src/services/ai.service.js
- backend/src/services/notification.service.js

MIDDLEWARE

- backend/src/middleware/auth.js
- backend/src/middleware/errorHandler.js
- backend/src/middleware/notFound.js

HOW THE APP WORKS END TO END

AUTH FLOW

Frontend:

- mobile/app/auth.tsx shows login/register UI
- mobile/lib/auth.tsx stores session in AsyncStorage
- mobile/lib/forum-api.ts sends auth API calls

Backend:

- backend/src/controllers/auth.controller.js handles register, login, OAuth completion, magic link completion, current profile, admin management, logout, password reset, profile update

Important behavior:

- local mobile session is stored in AsyncStorage
- API token is sent in Authorization header
- app refreshes session on app foreground

QUESTION FLOW

Create question:

- UI screen: mobile/app/(tabs)/explore.tsx
- API function: forumApi.createQuestion(...)
- backend controller: backend/src/controllers/questions.controller.js -> createQuestion
- data saved in MongoDB Question model

List questions:

- UI screen: mobile/app/(tabs)/index.tsx
- API function: forumApi.listQuestions()
- backend controller: questions.controller.js -> listQuestions

Open one question:

- UI screen: mobile/app/questions/[questionId].tsx
- API function: forumApi.getQuestion(questionId)
- backend controller: questions.controller.js -> getQuestionById

ANSWER FLOW

Post answer:

- UI screen: mobile/app/questions/[questionId].tsx
- API function: forumApi.createAnswer(questionId, payload)
- backend controller: backend/src/controllers/answers.controller.js -> createAnswer

List answers:

- same question thread screen
- backend controller: answers.controller.js -> listAnswers

COMMENT FLOW

Post comment on answer:

- UI screen: mobile/app/questions/[questionId].tsx
- API function: forumApi.createComment('answer', targetId, { body })
- backend controller: backend/src/controllers/comments.controller.js -> createComment

VOTE FLOW

Vote on question or answer:

- UI screens:
 - mobile/app/(tabs)/index.tsx
 - mobile/app/questions/[questionId].tsx
- API function: forumApi.castVote(targetType, targetId, value)
- backend controller: backend/src/controllers/votes.controller.js -> castVote

NOTIFICATION FLOW

Frontend:

- mobile/lib/notifications.tsx polls every 30 seconds
- mobile/app/(tabs)/inbox.tsx shows notifications

Backend:

- backend/src/controllers/notifications.controller.js
- backend/src/services/notification.service.js

Notifications are created when:

- answers are added
- comments are added

AI EXPLAIN FLOW

Frontend:

- mobile/app/(tabs)/explore.tsx
- user pastes code and taps Explain snippet

API wrapper:

- mobile/lib/forum-api.ts -> explainCode(...)

Backend:

- backend/src/controllers/ai.controller.js
- backend/src/services/ai.service.js

The backend:

- detects language
- chooses explanation mode
- tries Groq or OpenAI or Gemini
- returns structured explanation JSON

WHERE IMPORTANT BUTTONS ARE CODED

HOME FEED

File:

- mobile/app/(tabs)/index.tsx

Important buttons there:

- Ask Question button
- navigates to /(tabs)/explore
- Profile icon button
- navigates to /profile
- Filter button
- opens saved technology filters
- Feed tab buttons
- switch between Newest, Active, Unanswered
- Question card press
- opens /questions/[questionId]
- Vote buttons on question cards
- call forumApi.castVote('question', ...)

AUTH SCREEN

File:

- mobile/app/auth.tsx

Important buttons:

- Log in / Create account
- GitHub sign-in
- Google sign-in
- Email me a sign-in link
- Forgot password?

These connect to:

- forumApi.login
- forumApi.register
- forumApi.requestMagicLink
- forumApi.requestPasswordReset
- OAuth redirect helpers and Appwrite callback handling

ASK QUESTION + AI EXPLAIN SCREEN

File:

- mobile/app/(tabs)/explore.tsx

Important buttons:

- Publish
- creates a new question
- Explain snippet
- calls AI explain API
- History button
- opens AI history
- Clear button
- clears explanation result

QUESTION THREAD SCREEN

File:

- mobile/app/questions/[questionId].tsx

Important buttons:

- Vote up/down on question
- Post answer
- Vote up/down on each answer
- Reply button on answer comment box
- Delete question
- Delete answer
- Delete comment

This is one of the most important files if someone asks:

- where answer posting happens

- where comments happen
- where vote logic is triggered
- where the question detail UI is built

PROFILE SCREEN

File:

- mobile/app/profile.tsx

Important buttons:

- Edit profile
- Manage users and admins
- Delete question
- Delete answer
- Logout icon

EDIT PROFILE SCREEN

File:

- mobile/app/edit-profile.tsx

Important buttons:

- Change cover photo
- Change profile picture
- Remove image
- Save
- Cancel

This file handles:

- image picker
- Cloudinary upload
- profile save
- optional password change

INBOX SCREEN

File:

- mobile/app/(tabs)/inbox.tsx

Important buttons:

- Mark all read
- Open notification

MORE SCREEN

File:

- mobile/app/(tabs)/more.tsx

Important buttons:

- Theme toggle area
- Edit profile
- Ask a question
- Manage users
- Sign out

ADMIN SCREEN

File:

- mobile/app/admin.tsx

Important buttons:

- Search
- Make admin / Remove admin
- Disable account / Enable account

MAIN SHARED FRONTEND UTILITY FILES

MOBILE/LIB/FORUM-API.TS

This is one of the most important files in the whole app.

Why:

- almost every screen uses it
- it contains the API base URL detection
- it adds Authorization headers
- it defines all frontend-to-backend API calls

If someone asks:

"Where does the app call the backend?"

Start here.

MOBILE/LIB/AUTH.TSX

Controls:

- current user state
- login/register/logout
- session restore
- session refresh

MOBILE/LIB/NOTIFICATIONS.TSX

Controls:

- notification list state
- unread count
- polling
- mark read / mark all read

MOBILE/LIB/APPEARANCE.TSX

Controls:

- light/dark/system mode
- palette selection
- persisted theme mode

MAIN BACKEND FILES TO REMEMBER

BACKEND/SRC/CONTROLLERS/AUTH.CONTROLLER.JS

Use this file for:

- login
- register
- magic link
- OAuth
- password reset
- profile update
- admin role update
- disable user

BACKEND/SRC/CONTROLLERS/QUESTIONS.CONTROLLER.JS

Use this file for:

- list questions
- create question
- get question
- delete question

BACKEND/SRC/CONTROLLERS/ANSWERS.CONTROLLER.JS

Use this file for:

- list answers
- create answer
- delete answer

BACKEND/SRC/CONTROLLERS/COMMENTS.CONTROLLER.JS

Use this file for:

- list comments
- create comment
- delete comment

BACKEND/SRC/CONTROLLERS/VOTES.CONTROLLER.JS

Use this file for:

- question votes
- answer votes

BACKEND/SRC/CONTROLLERS/NOTIFICATIONS.CONTROLLER.JS

Use this file for:

- get notifications
- unread count
- mark read
- mark all read

BACKEND/SRC/SERVICES/AI.SERVICE.JS

Use this file for:

- AI prompt construction
- provider selection
- response parsing
- fallback behavior

QUICK "WHERE DO I EDIT..." CHEATSHEET

If you want to change:

- login screen UI -> mobile/app/auth.tsx
- home feed UI -> mobile/app/(tabs)/index.tsx
- ask question form -> mobile/app/(tabs)/explore.tsx
- question detail screen -> mobile/app/questions/[questionId].tsx
- profile UI -> mobile/app/profile.tsx
- edit profile form -> mobile/app/edit-profile.tsx
- inbox UI -> mobile/app/(tabs)/inbox.tsx
- more/settings UI -> mobile/app/(tabs)/more.tsx
- admin panel UI -> mobile/app/admin.tsx
- all API calls from app -> mobile/lib/forum-api.ts
- session/auth state -> mobile/lib/auth.tsx
- notification state -> mobile/lib/notifications.tsx
- theme logic -> mobile/lib/appearance.tsx
- backend server startup -> backend/server.js
- backend route wiring -> backend/src/routes/index.js
- auth backend logic -> backend/src/controllers/auth.controller.js
- question backend logic -> backend/src/controllers/questions.controller.js
- answer backend logic -> backend/src/controllers/answers.controller.js
- comment backend logic -> backend/src/controllers/comments.controller.js
- vote backend logic -> backend/src/controllers/votes.controller.js
- notification backend logic -> backend/src/controllers/notifications.controller.js
- AI backend logic -> backend/src/services/ai.service.js
- database connection -> backend/src/config/db.js
- env parsing -> backend/src/config/env.js

HOW TO EXPLAIN THIS PROJECT IN ONE SHORT ANSWER

You can describe it like this:

"This is a mobile Q&A app built with Expo and React Native on the frontend, and Node.js + Express + MongoDB on the backend. The mobile app uses Expo Router for navigation and a shared API wrapper to call backend routes. The backend handles auth, questions, answers, comments, votes, notifications, and an AI code explainer. Appwrite is used for OAuth, magic links, and password reset, while Cloudinary is used for profile image uploads."

BEST FILES TO LEARN FIRST

If you want to understand the app quickly, read these first in this order:

1. mobile/lib/forum-api.ts
2. mobile/app/_layout.tsx
3. mobile/app/(tabs)/index.tsx
4. mobile/app/(tabs)/explore.tsx
5. mobile/app/questions/[questionId].tsx
6. mobile/app/auth.tsx
7. backend/server.js
8. backend/src/routes/index.js
9. backend/src/controllers/auth.controller.js
10. backend/src/controllers/questions.controller.js
11. backend/src/controllers/answers.controller.js
12. backend/src/services/ai.service.js

FINAL NOTE

If someone asks you "where is this button coded?", usually answer it in 2 steps:

1. Find the screen file inside mobile/app or mobile/components
2. If the button calls the backend, trace it through mobile/lib/forum-api.ts into the matching backend controller

That pattern works for almost the whole project.